OS Lab Assignment 8 – Memory Management in xv6 (x86)

Tashya Aryan. Scope: Questions 1–4 (the PDF lists Q5–Q8 files only in the submission section, which is a misprint). Base: xv6-public (x86). All numbers below come from real runs under qemu-system-i386 (2 CPUs, 512 MB); raw output is in *evidence/*.

Q1 – Virtual to physical translation (va2pa)

Design. *sys_va2pa* calls *walkpgdir(pgdir, va, 0)* (made non-static and declared in defs.h). It returns -1 if the PDE/PTE is missing or *PTE_P* is clear, or if *PTE_U* is clear; otherwise it returns *PTE_ADDR(*pte)* | *(va & 0xFFF)* and prints PDX, PTX, offset and P/W/U. It never allocates (alloc=0), so querying a lazy page does not fault it in.

Result. global VA 0xC5C → PA 0xDEE8C5C; local 0x2FBC → 0xDEE5FBC; malloc block 0xAF98 → 0xDF76F98; 0x7FFFFFFF → -1 (page not present); 0x80000000 → -1 (present, U=0). After fork the global keeps VA 0xC5C but the parent has PA 0xDEE8C5C and the child 0xDF2CC5C, since xv6 copies pages on fork.

Q2 – Physical memory statistics (getmeminfo)

Design. *kmem* gets *nfree* and *total*. *kfree* increments and *kalloc* decrements *nfree* inside the existing *kmem.lock* critical section (the lock is off only during single-CPU boot). Every page freed by *kinit1/kinit2* is counted, and *total* is latched at the end of *kinit2*. *struct meminfo* (meminfo.h) holds total_pages, free_pages, user_pages = sz/PGSIZE, pt_pages, plus two extra fields: *pt_user_pages* and *resident_pages*. *vmcount()* walks the PDEs: pt_pages counts the page directory plus every present second-level table (a process owns 1 + 1 user + 64 kernel-half tables = 66 pages), resident_pages counts PTE_P entries below KERNBASE.

Stage	free_pages	sz/PGSIZE	resident	pt_pages
start	56789	3	3	66
after sbrk(64 pages), untouched (lazy)	56789	67	3	66
after touching all 64 pages	56725	67	67	66
child alive (zombie), before wait	56591	–	–	–
after wait()	56725	–	–	–

The fork cost 134 pages = 67 copied user pages + 66 page-table pages + 1 kernel stack, and after the child was reaped free_pages returned to 56725: **no leak**. (Total managed pages: 57000.) Note that user_pages (sz/PGSIZE) stops matching real usage once sbrk is lazy – hence the extra *resident_pages* field.

Q3 – Page-table dump (vmprint)

Design. *vmdump()* in vm.c prints the directory PA, then for each valid PDE below KERNBASE (PDX < 512) its index, page-table PA and flags, and for each valid PTE (indented one more level) the index, the mapped VA computed as *(pdx<<22)*|*(ptx<<12)*, the frame PA and P/W/U. Entries at or above KERNBASE are only counted (64 PDEs / 65536 PTEs). The guard page is any present PTE with U clear and is tagged in the output; a final line compares user-accessible pages with sz/PGSIZE.

```
===== 1. at start-up =====     sz=0x3000, 3 pages
 PDE[0]: pt_pa=0xdee7000 flags=P W U
```

```
    PTE[0]: va=0x0    pa=0xdee8000 flags=P W U
    PTE[1]: va=0x1000 pa=0xdee6000 flags=P W -   <-- guard page (P, no U)
    PTE[2]: va=0x2000 pa=0xdee5000 flags=P W U
summary: user-accessible pages=2  guard pages=1  sz/PGSIZE=3
after sbrk(8*4096)+touch: sz=0xb000: user-accessible=10, guard=1, sz/PGSIZE=11
after sbrk(-4096):        sz=0xa000: user-accessible= 9, guard=1, sz/PGSIZE=10
```

In every dump user-accessible pages = sz/PGSIZE − 1: the guard page is mapped (P=1, W=1) but U=0, so exec's *clearpteu* makes it count in sz but not as user-accessible. New PTEs 3..10 appear after sbrk and PTE 10 disappears after the shrink. With lazy sbrk (Q4) the test touches each new page, otherwise nothing would appear until first use.

Q4 – Lazy allocation

Design. *sys_sbrk*: for n ≥ 0 it only checks overflow / KERNBASE and raises *p->sz*. For n < 0 it rejects shrinking below the new field *p->heapstart* (set by exec to the end of the stack page, copied by fork) and then calls *growproc*, whose *deallocuvm* frees only pages that have PTE_P and skips holes. *trap()* handles *T_PGFLT* via *lazyalloc(rcr2())* in vm.c: the address must satisfy heapstart ≤ va < sz and have no present PTE; then it does *kalloc*, zero-fills and *mappages(..., PTE_W|PTE_U)* (so mappages was made non-static). Anything else prints a message and sets *killed* (the process is killed on return to user mode); kernel stays up. *copyuvm* skips missing PDEs/PTEs instead of panicking, and *uva2ka* tolerates a missing PTE. Because the kernel runs on the process's page table, a kernel access to an untouched buffer (read(), pipe read, getmeminfo) simply takes the same fault and is repaired transparently.

Step (lazytest)	free_pages	sz/PGSIZE	resident
start	56789	3	3
after sbrk(32 MB), untouched	56789	8195	3
after touching 5 pages 6 MB apart	56780	8195	8
after read()/pipe-read into untouched buffers	56776	8195	11
after sbrk(-32 MB)	56784	3	3
end (children killed)	56784	3	3

sbrk(32 MB) cost 0 frames. Touching 5 pages cost 9 frames (5 data + 4 new page tables because the pages are in different 4 MB regions). fork+exit+wait left free pages unchanged (56779 → 56779). Shrinking freed exactly the 8 pages that really existed. A child writing to 0x70000000 (≥ sz) and one writing to the guard page 0x1000 were each killed with a message while the parent continued. The stock *usertests* suite, run on the final kernel, ended with **ALL TESTS PASSED** (its sbrk test probes addresses ≥ KERNBASE, which now produce kill messages, as intended).

Key data structures and code (summary)

```
struct meminfo { total_pages, free_pages, user_pages, pt_pages,
                 pt_user_pages, resident_pages };          // meminfo.h
struct { ...; int nfree; int total; } kmem;                 // kalloc.c
struct proc { ...; uint sz; uint heapstart; ... };          // proc.h
syscalls: SYS_va2pa=22  SYS_getmeminfo=23  SYS_vmprint=24   // syscall.h
trap():  case T_PGFLT: if(lazyalloc(rcr2())==0) break;
         else if(user mode) { print msg; myproc()->killed = 1; } else panic
```

Files changed: vm.c (walkpgdir/mappages exported, copyuvm and uva2ka hardened, vmcount, vmdump, lazyalloc), kalloc.c (counters), sysproc.c (sbrk, va2pa, getmeminfo, vmprint), trap.c (T_PGFLT), proc.c/proc.h/exec.c (heapstart), syscall.c/.h, usys.S, user.h, defs.h, Makefile; new meminfo.h and the four test programs.

Problems encountered and fixes

1) Stock xv6 does not compile with the newer gcc (-Werror on mp.c): added -Wno-error. 2) *pte_t* is not visible in defs.h: used *uint** in the prototype. 3) Lazy allocation made sz/PGSIZE ≠ real usage, so the Q3 check and Q2 stats needed the guard-page reasoning and the extra *resident_pages* field. 4) Knowing where lazy pages are legal: used the new *heapstart* field so faults below the stack and in the guard page are rejected. 5) Hand-computing free-page deltas (e.g. 134 for fork, 9 for five far-apart touches) exposed the 64 kernel-half page tables each xv6 process carries.

Limitations

A lazy fault taken *inside the kernel* that fails (out of memory) panics, since the kernel cannot unwind mid-copy; user-mode failures only kill the process. Page-table pages are not freed on shrink (stock behaviour) but are freed at exit. Screenshots must be captured by the student (see screenshots/CHECKLIST.md).